

Generador de presupuestos y archivos con Python

Descripción

Este proyecto fue desarrollado para el Trabajo Práctico de Programación de la carrera Soporte de Infraestructura.

El objetivo principal es demostrar cómo Python puede utilizarse para automatizar la generación de distintos tipos de archivos a partir de una misma carga de datos. El programa permite ingresar productos, cantidades y costos unitarios para generar automáticamente documentación asociada a una venta o presupuesto.

A partir de una única operación, el sistema genera distintos archivos con finalidades diferentes:

- Un archivo Excel para control interno o inventario.
- Un documento Word con información de costos.
- Un presupuesto en PDF para el cliente.
- Un PDF final consolidado que incorpora condiciones legales o anexos comerciales.

La idea general es mostrar cómo Python puede reducir tareas repetitivas, evitar la carga duplicada de información y organizar la documentación de forma más clara para distintos destinatarios.

Integrantes

- Federico Flores
- Leandro Demaria

Tema del Trabajo Práctico

Tema principal de Python: Generación de archivos.

Librerías utilizadas:

- openpyxl
- python-docx
- ReportLab
- PyPDF2

Tema complementario de Inteligencia Artificial:

- LLMs como herramienta de desarrollo.

Objetivo del proyecto

El proyecto busca representar un caso práctico de automatización documental. A partir de la carga de productos para una venta o presupuesto, Python procesa los datos, realiza cálculos y genera distintos documentos automáticamente.

Este tipo de solución puede aplicarse en contextos administrativos, comerciales o técnicos donde una misma operación requiere documentación para diferentes usos, por ejemplo:

- Control de stock o inventario.
- Registro interno de costos.
- Presupuesto para el cliente.
- Consolidación de documentación comercial.

Funcionamiento general

El programa solicita al usuario que ingrese la cantidad de productos que desea cargar. Luego, para cada producto, pide los siguientes datos:

- Descripción del producto.
- Cantidad.
- Costo unitario.

Con esa información, el sistema calcula:

- Costo total.
- Ganancia.
- Precio sin IVA.
- IVA.
- Precio final.
- Total del presupuesto.

Una vez finalizada la carga, el programa genera automáticamente los archivos correspondientes dentro de la carpeta salida.

Flujo de trabajo

El flujo general del proyecto es el siguiente:

text Carga de productos por consola

↓

Cálculo de costos, ganancia, IVA y precio final

↓

Generación de archivo Excel con openpyxl

↓

Generación de documento Word con python-docx

↓

Generación de presupuesto PDF con ReportLab

↓

Unión del PDF con condiciones legales usando PyPDF2

↓

Presupuesto final consolidado

Archivos generados

El programa genera los siguientes archivos:

inventario.xlsx -> Planilla Excel con información de productos y cantidades. ->Control interno, inventario o stock.

costos.docx -> Documento Word con información de costos, ganancia y totales. -> Administración, ventas o contabilidad.

presupuesto.pdf -> Presupuesto generado en formato PDF ->Documento para entregar al cliente.

presupuesto_final.pdf -> PDF final que une el presupuesto con condiciones legales -> Entrega final consolidada.

Librerías utilizadas

openpyxl

openpyxl es una librería de Python que permite crear, leer y modificar archivos Excel con extensión .xlsx.

En este proyecto se utiliza para generar el archivo:

text inventario.xlsx

Este archivo puede servir como salida interna para control de productos, cantidades o inventario.

python-docx

python-docx permite crear y modificar documentos de Microsoft Word desde Python.

En este proyecto se utiliza para generar el archivo:

text costos.docx

Este documento contiene información interna del presupuesto, como costos, ganancias y totales.

ReportLab

ReportLab es una librería que permite generar archivos PDF desde Python.

En este proyecto se utiliza para crear el archivo:

text presupuesto.pdf

Este archivo representa el presupuesto en un formato más formal, pensado para ser entregado al cliente.

PyPDF2

PyPDF2 permite manipular archivos PDF existentes. Entre sus funciones principales se encuentra la posibilidad de unir varios PDFs en un único documento.

En este proyecto se utiliza para generar:

text presupuesto_final.pdf

Este archivo surge de unir el presupuesto generado con un anexo de condiciones legales o comerciales.

Requisitos previos

Para ejecutar el proyecto es necesario tener instalado:

- Python 3.
- pip, el gestor de paquetes de Python.
- Las librerías indicadas en el archivo requirements.txt.

Para verificar si Python está instalado, se puede ejecutar:

bash python --version

En algunos sistemas Windows también puede utilizarse:

bash py --version

Para verificar si pip está instalado:

bash pip --version

Instalación de dependencias

Opción 1: instalar todas las dependencias desde requirements.txt

La forma recomendada es instalar todas las librerías necesarias desde el archivo requirements.txt.

Desde la terminal, ubicarse en la carpeta del proyecto y ejecutar:

```
bash pip install -r requirements.txt
```

En algunos sistemas Windows puede utilizarse:

```
bash py -m pip install -r requirements.txt
```

Opción 2: instalar cada librería por separado

También se pueden instalar las librerías individualmente.

Instalar openpyxl:

```
bash pip install openpyxl
```

Instalar python-docx:

```
bash pip install python-docx
```

Instalar ReportLab:

```
bash pip install reportlab
```

Instalar PyPDF2:

```
bash pip install PyPDF2
```

En Windows, si el comando pip no funciona, se puede usar:

```
bash py -m pip install openpyxl py -m pip install python-docx py -m pip install reportlab py -m pip install PyPDF2
```

Archivo requirements.txt recomendado

El archivo requirements.txt debería contener únicamente las dependencias necesarias para ejecutar el proyecto:

```
text openpyxl python-docx reportlab PyPDF2
```

Ejecución del programa

Para ejecutar el programa, abrir una terminal en la carpeta del proyecto y utilizar:

```
bash python Carga_de_productos.py
```

En algunos sistemas Windows también puede ejecutarse con:

```
bash py Carga_de_productos.py
```

Luego, el programa solicitará la carga de productos por consola.

Ejemplo de datos solicitados:

```
text ¿Cuántos productos desea ingresar? Descripción del producto: Cantidad: Costo unitario:
```

Al finalizar la carga, se generarán automáticamente los archivos dentro de la carpeta salida.

Descripción del archivo principal

El archivo principal del proyecto es:

```
text Carga_de_productos.py
```

Este script concentra el funcionamiento principal del programa:

1. Solicita los datos al usuario.
2. Calcula costos, ganancia, IVA y precio final.
3. Genera un Excel con información de inventario.
4. Genera un Word con información de costos.
5. Genera un PDF con el presupuesto.
6. Une el presupuesto PDF con un archivo de condiciones legales.
7. Guarda los resultados en la carpeta salida.

Relación con Inteligencia Artificial

Como tema complementario del trabajo práctico, se aborda el uso de LLMs como herramienta de desarrollo.

Un LLM, o modelo de lenguaje grande, es una herramienta de inteligencia artificial capaz de comprender instrucciones en lenguaje natural y generar respuestas en forma de texto, código, explicaciones o documentación.

En este proyecto, los LLMs pueden ser útiles para acompañar el desarrollo de distintas maneras:

- Ayudar a comprender el uso de librerías nuevas.
- Generar ejemplos de código.
- Explicar errores.
- Sugerir mejoras de estructura.
- Redactar comentarios.

- Documentar funciones.
- Ayudar a escribir el archivo README.md.
- Mejorar la redacción de textos técnicos.

La idea principal es que el LLM no reemplaza al programador. El programador define la lógica, comprende el problema, revisa el código y valida los resultados. El LLM funciona como un asistente que puede acelerar tareas de escritura, revisión, explicación y documentación.

Herramientas de IA relacionadas

Algunas herramientas basadas en LLMs que pueden utilizarse como apoyo al desarrollo son:

- ChatGPT.
- GitHub Copilot.
- Cursor.
- Claude.
- Gemini Code Assist.
- Replit AI.
- Amazon Q Developer.

Estas herramientas pueden asistir en tareas como generación de código, explicación de errores, documentación, refactorización y análisis de fragmentos de programas.

Buenas prácticas al usar LLMs en programación

Al utilizar LLMs como apoyo al desarrollo, es importante tener en cuenta algunas buenas prácticas:

- Dar instrucciones claras y concretas.
- Explicar el contexto del proyecto.
- Revisar siempre el código generado.
- Probar el programa antes de usarlo.
- No copiar código sin entenderlo.
- No compartir datos sensibles.
- Verificar que las librerías y funciones sugeridas existan realmente.
- Usar la IA como apoyo al aprendizaje, no como reemplazo.

Conclusión

Este proyecto permite demostrar cómo Python puede automatizar la generación de documentación a partir de una misma carga de datos.

Mediante el uso de librerías específicas, el programa genera archivos Excel, Word y PDF, además de consolidar documentos finales con anexos. Esto permite representar un flujo simple pero útil de automatización documental aplicado a una venta o presupuesto.

Además, el trabajo incorpora el uso de LLMs como herramienta complementaria de desarrollo, mostrando cómo la inteligencia artificial puede ayudar a escribir, revisar, explicar y documentar código de manera más ágil.

Python se encarga de ejecutar la automatización. Los LLMs acompañan el proceso de desarrollo y documentación.